



10. Python: Lists

"Ordered, Changeable, and Versatile"

Previous Section:

[!\[\]\(c3d993ca47bfe2a953c700506ce31fa0_img.jpg\) 9. Python: Local and Global Variables](#)

Contents:

[1. What is a List?](#)

[Properties of Lists](#)

[2. Indexing and Slicing in Lists](#)

[Indexing in Lists](#)

[Slicing in Lists](#)

[Creating and Modifying Lists](#)

[Creating Lists using range\(\)](#)

[Slicing with Generated Lists](#)

[3. Operators in Lists](#)

[Summary](#)

[References](#)

Lists are one of the most important data structures in both Programming and Artificial Intelligence. Whenever we work with large amounts of data, collections of values, datasets, or even Machine Learning features, lists are often one of the first tools we use.

Think about it for a moment. A single variable can only store one value at a time. But what if we want to store:

- A list of student scores

- A collection of names
- Thousands of image pixels
- A sequence of predictions from a Machine Learning model

Storing them one by one would become impossible to manage.

This is where lists become extremely useful. A list allows us to store multiple values together inside a single structure, while still keeping the order of those values. And because lists are mutable, we can continuously modify them by adding, removing, or changing elements whenever we want.

In Python, lists are everywhere. From beginner programs, to advanced AI systems, lists become one of the fundamental building blocks of programming itself.

1. What is a List?

A list is a sequence that can store multiple values inside a single variable. One of the most powerful things about lists is that they can store values of different data types at the same time.

For example:

```
data = [1, "Iris", 3.14, True]
print(data)
```

```
[1, 'Iris', 3.14, True]
```

Here, the list stores: Integer; String; Float; Boolean, all together inside one structure.

We can also create a list using the `list()` constructor. Example:

```
numbers = list((1, 2, 3, 4, 5))
print(numbers)
```

```
[1, 2, 3, 4, 5]
```

This method is useful when converting other iterable objects into a list.

Properties of Lists

- **Lists are Mutable:** This means we can modify the contents of the list without creating a completely new list. We can insert elements, delete elements, replace elements

Example:

```
numbers = [1, 2, 3]

# Replace element
numbers[1] = 10

print(numbers)
```

[1, 10, 3]

The list itself remains the same object, but its contents changed.

- **Lists Preserve Order:** Every element inside a list has a position called an index. Python indexing starts from 0.

Example:

```
fruits = ["Apple", "Banana", "Orange"]

print(fruits[0])
print(fruits[1])
print(fruits[2])
```

Apple
Banana
Orange

So the indexes are:

Element	Index
Apple	0
Banana	1
Orange	2

- **Lists Use Square Brackets:** Lists in Python are created using square brackets `[]`.

Example:

```
numbers = [1, 2, 3, 4, 5]

print(numbers)
```

[1, 2, 3, 4, 5]

2. Indexing and Slicing in Lists

Indexing in Lists

Indexing allows us to retrieve a specific element from the list. Syntax:

`list[index]`

Example:

```
numbers = [10, 20, 30, 40, 50]

print(numbers[0])
print(numbers[3])
```

10

40

Python also allows negative indexing. This means:

- `-1` → last element
- `-2` → second last element
- and so on.

Example:

```
numbers = [10, 20, 30, 40, 50]

print(numbers[-1])
print(numbers[-2])
```

50
40

This is extremely useful when we want to access elements from the end of the list.

Slicing in Lists

Slicing allows us to retrieve multiple elements from a list.

Syntax:

`list[start : stop : step]`

Where: `start` → starting index; `stop` → stopping index (exclusive); `step` → movement step

- Basic Slicing

```
numbers = [1, 2, 3, 4, 5]

print(numbers[1:4])
```

[2, 3, 4]

Notice something important: **The stop index is NOT included.**

- Using Step in Slicing

```
numbers = [1, 2, 3, 4, 5, 6, 7, 8]

print(numbers[::2])
```

[1, 3, 5, 7]

The list moves two steps at a time.

- Reversing a List using Slicing - One of the coolest tricks in Python:

```
numbers = [1, 2, 3, 4, 5]

print(numbers[::-1])
```

[5, 4, 3, 2, 1]

The negative step reverses the traversal direction.

Creating and Modifying Lists

Example:

```

# Creating a list
numbers = [1, 2, 3, 4, 5]

# Indexing and Slicing
print('Element at position 0:', numbers[0])
print('Element at position 3:', numbers[3])
print('Last element:', numbers[-1])
print('Element from 1 to 3:', numbers[1:4])

# Modifying the list
numbers.append(6)
print('Insert 6:', numbers)

numbers[2] = 10
print('Replace 10:', numbers)

```

Element at position 0: 1
 Element at position 3: 4
 Last element: 5
 Element from 1 to 3: [2, 3, 4]

Insert 6: [1, 2, 3, 4, 5, 6]
 Replace 10: [1, 2, 10, 4, 5, 6]

Here we learned two important operations:

- `append()` → insert new element at the end
- Direct indexing assignment → replace an element

Creating Lists using `range()`

Sometimes manually typing every number becomes inefficient.

Instead, we can generate lists automatically using `range()`. Example:

```
numbers = [i for i in range(1, 11)]  
print(numbers)
```

[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

This is called a List Comprehension. We will study it more deeply later, but for now, just understand that it creates lists automatically.

Slicing with Generated Lists

Example:

```
numbers = [i for i in range(1, 11)]  
  
print('Element from 2 to 5:', numbers[2:6])  
print('Element from 1 to 10:', numbers[1:])  
print('The last element:', numbers[-1])
```

Element from 2 to 5: [3, 4, 5, 6]

Element from 1 to 10: [2, 3, 4, 5, 6, 7, 8, 9, 10]

The last element: 10

3. Operators in Lists

Lists also support several useful operators.

- Concatenation (+): Combines two lists together.

```
list1 = ['Apple', 'Microsoft', 'Google']  
list2 = ['Facebook', 'Amazon', 'Tesla']  
  
print(list1 + list2)
```

['Apple', 'Microsoft', 'Google', 'Facebook', 'Amazon', 'Tesla']

- Repetition (`*`): Repeats the list multiple times.

```
list1 = ['Apple', 'Microsoft', 'Google']  
  
print(list1 * 2)
```

['Apple', 'Microsoft', 'Google', 'Apple', 'Microsoft', 'Google']

- Reverse using Slicing

```
list1 = ['Apple', 'Microsoft', 'Google']  
  
print(list1[::-1])
```

['Google', 'Microsoft', 'Apple']

Summary

Lists are not just beginner concepts. They are fundamental structures used throughout Computer Science, Data Science, and Artificial Intelligence.

Once we fully understand lists, many advanced concepts become easier later: Matrices; DataFrames; Feature vectors; Neural network inputs; Dataset handling; Graph structures

So while lists may look simple at first, they are actually one of the most important foundations in Python programming.

References

<https://www.geeksforgeeks.org/python/python-lists/>

<https://developers.google.com/edu/python/lists>

Next Sections:

[`</>` 10.1. Python: Common Methods in Lists](#)

[`</>` 10.2. Python: List Comprehension](#)

[`</>` 11. Python: Tuples](#)